

Hierarchical DivNoising (HDN) - 3D Implementation

Complete Technical Documentation

Branch: `3d_initial`

Project: Interpretable Unsupervised Diversity Denoising and Artefact Removal

Authors: Mangal Prakash, Mauricio Delbracio, Peyman Milanfar, Florian Jug

Table of Contents

1. [Executive Summary](#)
2. [Architecture Overview](#)
3. [Core Components](#)
 - 3.1 [Ladder VAE Model](#)
 - 3.2 [Layer Components](#)
 - 3.3 [Stochastic Blocks](#)
 - 3.4 [Likelihood Functions](#)
4. [Training Pipeline](#)
 - 4.1 [Training Loop](#)
 - 4.2 [Forward Pass & Loss Functions](#)
 - 4.3 [Boilerplate Utilities](#)
5. [Data Management](#)
 - 5.1 [Dataset Classes](#)
 - 5.2 [Samplers](#)
 - 5.3 [Data Loading](#)
6. [Utility Functions](#)
7. [3D Training Example](#)
8. [Configuration Parameters](#)
9. [Mathematical Foundations](#)
10. [File Structure](#)

1. Executive Summary

This codebase implements **Hierarchical DivNoising (HDN)**, a state-of-the-art deep learning approach for image denoising and artefact removal. The implementation extends the original 2D framework to support **3D volumetric data**, making it suitable for applications in:

- Medical imaging (CT, MRI)
- Microscopy (confocal, electron microscopy)
- Biological imaging (cell segmentation, organelle detection)

Key Features

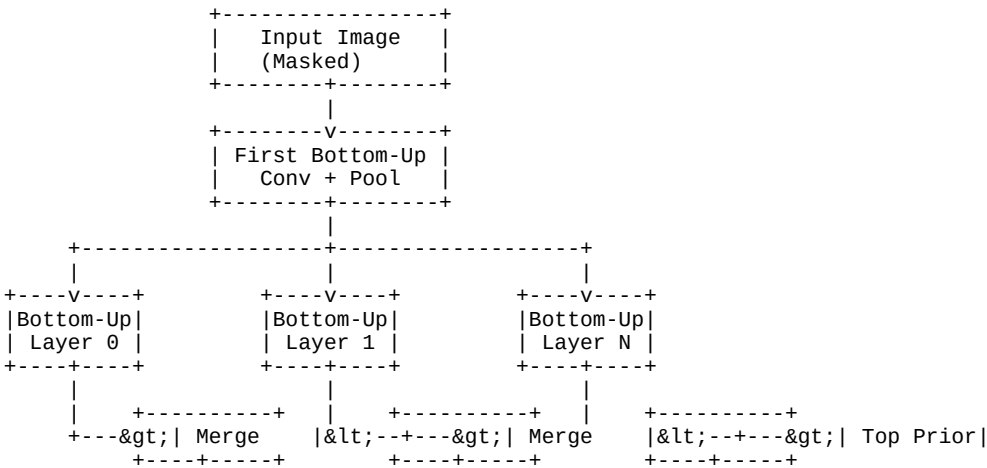
Feature	Description
Hierarchical VAE	Multi-level latent variable model for diverse denoising
2D/3D Support	Flexible architecture supporting both 2D and 3D convolutions
Semi-supervised Learning	Hybrid training with labeled and unlabeled data
Contrastive Learning	Optional contrastive loss for improved feature learning
Mixture Models	Gaussian Mixture Model priors for complex distributions
Noise Models	Support for Gaussian and histogram-based noise models

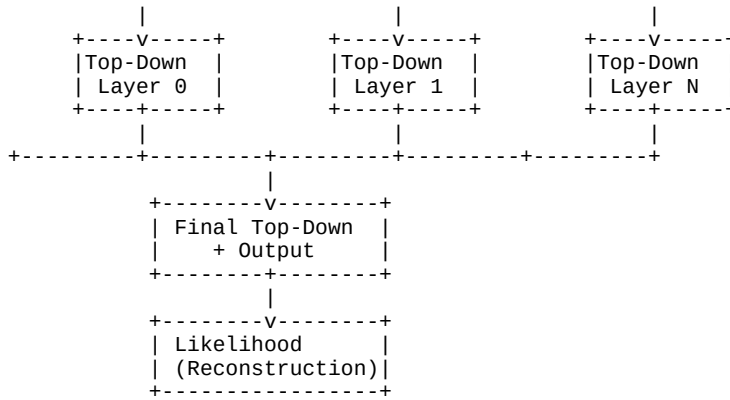
Technology Stack

- **Framework:** PyTorch
- **Mixed Precision:** Automatic Mixed Precision (AMP) with GradScaler
- **Experiment Tracking:** Weights & Biases (wandb)
- **Data Format:** TIFF images (tiff file)

2. Architecture Overview

The HDN architecture is based on a **Ladder Variational Autoencoder (Ladder VAE)**, which employs a hierarchical latent structure where information flows bidirectionally between multiple abstraction levels.





Inference Flow

1. **Bottom-Up Pass:** Encodes input through hierarchical deterministic residual blocks
2. **Top-Down Pass:** Samples latent variables and generates reconstruction
3. **Merge Layers:** Combine bottom-up and top-down information
4. **Likelihood:** Computes reconstruction probability

3. Core Components

3.1 Ladder VAE Model (`models/lvae.py`)

The **LadderVAE** class is the main model implementation.

Constructor Parameters

```
class LadderVAE(nn.Module):
    def __init__(
        self,
        z_dims,          # List[int]: Latent dimensions per layer [32, 32, 32]
        device,          # torch.device: CUDA or CPU
        data_mean,       # float: Dataset mean for normalization
        data_std,        # float: Dataset std for normalization
        color_ch=1,      # int: Number of input channels
        noiseModel=None, # NoiseModel: Custom noise model
        blocks_per_layer=5, # int: Residual blocks per layer
        conv_mult=3,     # int: 2 for 2D, 3 for 3D convolutions
        nonlin=nn.ELU,   # Activation function
        merge_type="residual", # str: How to merge BU and TD paths
        batchnorm=True,  # bool: Use batch normalization
        stochastic_skip=True, # bool: Skip connections around stochastic layers
        n_filters=64,    # int: Base number of filters
        dropout=0.2,     # float: Dropout probability
        free_bits=0.0,    # float: Free bits for KL (prevents collapse)
        learn_top_prior=False, # bool: Learnable top-level prior
        img_shape=None,  # Tuple: Input image shape
        res_block_type="bacdbacd", # str: Residual block structure
        gated=True,      # bool: Use gating mechanism
        grad_checkpoint=False, # bool: Gradient checkpointing
        no_initial_downscaling=True,
        analytical_kl=True, # bool: Analytical vs MC KL
        mode_pred=False,   # bool: Prediction mode (no sampling)
        use_uncond_mode_at=[], # List: Layers for unconditional mode
        mask_size=5,      # int: Inpainting mask size
        contrastive_learning=False, # bool: Enable contrastive loss
        margin=50,        # float: Contrastive margin
        lambda_contrastive=0.5, # float: Contrastive loss weight
        stochastic_block_type="normal", # str: "normal" or "mixture"
        conditional=False, # bool: Conditional generation
        condition_type='mlp', # str: Conditioning network type
    ):
```

```

        n_components=4,          # int: GMM components
        training_mode='supervised', # str: Training mode
        labeled_ratio=0.75,      # float: Ratio of labeled samples
    )

```

Key Methods

Method	Description
<code>`forward(x, y, x_orig, threshold)`</code>	Full forward pass with inference and sampling
<code>`bottomup_pass(x)`</code>	Encode input through bottom-up layers
<code>`topdown_pass(label, bu_values, ...)`</code>	Generate reconstruction from latents
<code>`sample_prior(n_imgs, ...)`</code>	Generate samples from prior
<code>`pad_input(x, dim)`</code>	Pad input to power-of-2 dimensions
<code>`increment_global_step()`</code>	Increment training step counter
<code>`update_mode(mode)`</code>	Switch training mode

Forward Pass Output Dictionary

```

output = {
    "ll": ll,          # Log-likelihood
    "z": td_data["z"], # List of sampled latents per layer
    "mu": td_data["mu"], # List of posterior means
    "kl": kl,          # KL divergence
    "cl": cl,          # Contrastive loss
    "logp": logprob_p, # Log probability under prior
    "out_mean": mean,  # Reconstruction mean
    "out_mode": mode,  # Reconstruction mode
    "out_sample": sample, # Sampled reconstruction
    "likelihood_params": params, # Likelihood parameters
    "ce": cross_entropy, # Cross-entropy (for classification)
    "entropy": entropy, # Entropy term
    "pi": mixing_weights, # GMM mixing weights
    "q": quadrants,      # Contrastive learning quadrants
}

```

3.2 Layer Components (`models/lvae_layers.py`)

TopDownLayer

Handles the generative (decoder) path with stochastic sampling.

```

class TopDownLayer(nn.Module):
    """
    Architecture when doing inference:
    p_params = output of top-down layer above
    bu = inferred bottom-up value at this layer
    q_params = merge(bu, p_params)
    z = stochastic_layer(q_params)
    [optional skip connection]
    top-down deterministic ResNet
    """

```

Key Features:

- Stochastic sampling (Normal or GMM)
- KL divergence computation
- Skip connections

- Learnable top-level prior

BottomUpLayer

Handles the inference (encoder) path.

```
class BottomUpLayer(nn.Module):
    """
    Bottom-up deterministic layer for inference.
    Sequence of residual blocks with optional downsampling.
    """
```

ResBlockWithResampling

Flexible residual block supporting up/downsampling.

```
class ResBlockWithResampling(nn.Module):
    """
    Supports both top-down (upsampling) and bottom-up (downsampling) modes.
    Uses strided convolutions for resampling.
    """
```

BlurPool

Anti-aliasing downsampling to reduce artifacts.

```
class BlurPool(nn.Module):
    """
    Applies Gaussian blur before downsampling to prevent aliasing.
    Kernel: [1, 2, 1] outer product, normalized.
    """
    def __init__(self, channels, stride=2):
        # Low-pass filter approximating Gaussian
        kernel = torch.tensor([1, 2, 1], dtype=torch.float32)
        kernel = kernel[:, None] * kernel[None, :]
        kernel = kernel / kernel.sum()
```

MergeLayer

Combines bottom-up and top-down information.

```
class MergeLayer(nn.Module):
    """
    Merge types:
    - "linear": Simple 1x1 convolution
    - "residual": 1x1 conv + ResidualGatedBlock
    """
    def forward(self, x, y):
        x = torch.cat((x, y), dim=1)
        return self.layer(x)
```

3.3 Stochastic Blocks (`lib/stochastic.py`)

The `StochasticConvBlock` handles latent variable sampling and KL computation.

Block Types

Type	Description
"normal"	Standard Gaussian latent variables
"mixture"	Gaussian Mixture Model with K components

Conditioning Methods

Method	Description
`"mlp"`	MLP-based conditioning network
`"transformer"`	Transformer-based feature extraction

Key Computations

```
# Posterior parameters q(z|x)
q_mu, q_lv = self.conv_in_q(q_params).chunk(2, dim=1)
q_std = torch.where(q_lv < 0, (q_lv / 2).exp(), 1 + q_lv)
q = Normal(q_mu, q_std)

# Reparameterized sampling
z = q.rsample()

# KL divergence
kl = kl_divergence(q, p)
```

Gumbel-Softmax for Discrete Variables

For mixture models with categorical latents:

```
y = F.gumbel_softmax(qy_logits, tau=self.temperature, hard=False)
self._update_temperature() # Annealing: max(0.5, T * 0.999)
```

Jensen-Shannon Divergence Regularization

```
def _compute_js_div(self, y):
    m = 0.5 * (y + self.prior_probs)
    js_div = 0.5 * torch.sum(y * torch.log(y / (m + 1e-10)), dim=1) + \
        0.5 * torch.sum(self.prior_probs * torch.log(self.prior_probs / (m + 1e-10)), dim=1)
    return js_div.mean()
```

3.4 Likelihood Functions (`lib/likelihoods.py`)

GaussianLikelihood

Simple Gaussian reconstruction likelihood.

```
class GaussianLikelihood(LikelihoodModule):
    def log_likelihood(self, x, params):
        logprob = -0.5 * (params["mean"] - x) ** 2
        return logprob
```

NoiseModelLikelihood

Uses a custom noise model (e.g., histogram-based or GMM noise model).

```
class NoiseModelLikelihood(LikelihoodModule):
    def log_likelihood(self, x, params):
        # Denormalize predictions
        predicted_s_denormalized = params["mean"] * self.data_std + self.data_mean
        x_denormalized = x * self.data_std + self.data_mean

        # Query noise model
        likelihoods = self.noiseModel.likelihood(x_denormalized, predicted_s_denormalized)
        return torch.log(likelihoods)
```

4. Training Pipeline

4.1 Training Loop (`training.py`)

The main training function orchestrates the training process.

```
def train_network(
    model,                # LadderVAE model
    lr,                   # Learning rate
    max_epochs,           # Number of epochs
    train_loader,         # Training DataLoader
    val_loader,           # Validation DataLoader
    gaussian_noise_std,   # Gaussian noise std (if applicable)
    model_name,           # Name for saving
    directory_path,       # Save directory
    batch_size=8,         # Batch size
    alpha=1,              # Reconstruction loss weight
    beta=1,               # KL loss weight
    gamma=1,              # Contrastive loss weight
    max_grad_norm=None,   # Gradient clipping
    amp=True,             # Mixed precision training
    gradient_scale=8192,  # AMP gradient scale
    use_wandb=True,       # Log to W&B
    initial_label_size=1, # Starting label size
    final_label_size=10,  # Final label size
    initial_mask_size=1,  # Starting mask size
    final_mask_size=10,   # Final mask size
    step_interval=20,     # Scheduler step interval
)
```

Training Modes

Mode	Description
"supervised"	All samples have labels
"semisupervised"	Mix of labeled and unlabeled samples
"unsupervised"	No labels, purely generative learning

Mode Switching Logic

```
if patience_ == 20 and train_loader.dataset.mode == "supervised":
    # Switch to semi-supervised after supervised plateau
    train_loader.dataset.set_mode('semisupervised')
    model.update_mode("semisupervised")

if patience_ == 20 and train_loader.dataset.radius < 10:
    # Increase sampling radius for pseudo-labels
    train_loader.dataset.increase_radius()
```

Curriculum Learning with Schedulers

```
class LabelSizeScheduler:
    """Gradually increases label/mask size during training."""
    def get_label_size(self, current_step):
        if self.mode == "step":
            direction = 2 if self.final_size > self.initial_size else -2
            intervals = current_step // self.step_interval
            new_size = self.initial_size + intervals * direction
            return clamp(new_size, self.initial_size, self.final_size)
```

4.2 Forward Pass & Loss Functions

Input Masking (Inpainting)

The model uses blind-spot masking for self-supervised learning:

```
def mask_input(x, model):
    x_masked = x.clone()
```

```

patch_size = x.shape[-1]
mask_size = model.mask_size
begin = (patch_size - mask_size) // 2
end = begin + mask_size

if model.conv_mult == 2: # 2D
    x_masked[:, :, begin:end, begin:end] = 0
elif model.conv_mult == 3: # 3D
    x_masked[:, :, begin:end, begin:end, begin:end] = 0
return x_masked

```

Loss Computation

```

# Total loss
loss = alpha * inpainting_loss + beta * kl_loss + gamma * cl_loss + ce + entropy

# Inpainting loss: Only computed on the masked center region
inpainting_loss = get_centre(recons_sep, patch_size, mask_size, conv).mean()

# KL loss: Sum across layers
kl_loss = torch.stack(td_data["kl"]).sum(0)
if free_bits > 0:
    kl = free_bits_kl(kl, free_bits)

# Contrastive loss
cl_loss = compute_cl_loss(mus=td_data["mu"], labels=td_data["pseudo_labels"])

```

Contrastive Learning Loss (`lib/utlis.py`)

Multi-scale contrastive loss using descriptors from all hierarchical levels:

```

def multiscale_cl(mus, labels, margin=1.5):
    B = len(mus[0])
    device = mus[0].device
    labels = labels[2].view(-1)

    # Build masks
    same = labels.unsqueeze(0).eq(labels.unsqueeze(1)) # Same class
    pos_mask = same & ~torch.eye(B, dtype=torch.bool) # Same, not self
    neg_mask = ~same # Different class

    # Multi-scale descriptors
    descriptors = torch.cat([
        F.adaptive_avg_pool2d(mus[i], (1, 1)).squeeze(-1).squeeze(-1)
        for i in range(len(mus))
    ], dim=1)
    descriptors = F.normalize(descriptors, dim=1)

    # Pairwise distances
    dist = torch.cdist(descriptors, descriptors, p=2)

    # Losses
    pos_d = dist[pos_mask]
    neg_d = dist[neg_mask]
    pos_loss = (pos_d ** 2).mean() # Pull together
    neg_loss = (F.relu(margin - neg_d) ** 2).mean() # Push apart

    return pos_loss + neg_loss

```

4.3 Boilerplate Utilities (`boilerplate/boilerplate.py`)

Optimizer Setup

```

def _make_optimizer_and_scheduler(model, lr, weight_decay):
    optimizer = optim.Adamax(model.parameters(), lr=lr, weight_decay=weight_decay)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, "min", patience=10, factor=0.9, min_lr=1e-12
    )
    return optimizer, scheduler

```


Prediction Functions

```
def predict_mmse(img_n, num_samples, model, gaussian_noise_std, device):
    """Minimum Mean Square Error estimate via sample averaging."""
    samples = []
    for j in range(num_samples):
        sample = predict_sample(image_sample, model, gaussian_noise_std, device)
        samples.append(np.squeeze(sample))
    return np.mean(np.array(samples), axis=0)
```

Test-Time Augmentation (TTA)

```
def tta_forward(x):
    """8-fold augmentation: rotations (0, 90, 180, 270) + horizontal flips."""
    x_aug = [x, np.rot90(x, 1), np.rot90(x, 2), np.rot90(x, 3)]
    x_aug_flip = x_aug.copy()
    for x_ in x_aug:
        x_aug_flip.append(np.fliplr(x_))
    return x_aug_flip

def tta_backward(x_aug):
    """Inverse TTA and average."""
    x_deaug = [
        x_aug[0], np.rot90(x_aug[1], -1), np.rot90(x_aug[2], -2),
        np.rot90(x_aug[3], -3), np.fliplr(x_aug[4]), ...
    ]
    return np.mean(x_deaug, 0)
```

5. Data Management

5.1 Dataset Classes (*'boilerplate/dataloader.py'*)

Custom3DDataset

Extracts 3D patches from volumetric data with label-based organization.

```
class Custom3DDataset(Dataset):
    def __init__(self, images, labels, patch_size=(64, 64, 64), mask_size=(5, 5, 5)):
        """
        Parameters:
        - images: List of 3D numpy arrays
        - labels: List of corresponding label maps
        - patch_size: (D, H, W) patch dimensions
        - mask_size: (D, H, W) center mask for inpainting
        """
```

Patch Extraction Logic:

```
for z in range(0, depth, d_stride):
    for y in range(0, height, h_stride):
        for x in range(0, width, w_stride):
            patch = img[z:z+d_patch, y:y+h_patch, x:x+w_patch]
            center_label = patch_label[z_center, y_center, x_center]

            if center_label != -1: # Valid label
                self.all_patches.append((patch, center_label, patch_label))
                patches_by_label[center_label].append(idx)
```

SemisupervisedDataset

Supports anchor + neighbor sampling for semi-supervised learning.

```
class SemisupervisedDataset(Dataset):
    """
    Groups: anchor (labeled) + neighbors (potentially unlabeled)
    Modes: "supervised" | "semisupervised"
    """
    def __getitem__(self, idx):
```

```

if self.mode == "supervised":
    # Return single labeled patch
    return patch, label, segment, coords
else:
    # Return anchor + 7 neighbors
    return patches, labels, segments, coords

```

Neighbor Sampling:

```

def _sample_neighbors(self, cy, cx, H, W, used_coords, k=7, max_tries=100):
    """Sample k neighbors within radius disk."""
    neighbors = []
    while len(neighbors) < k and max_tries:
        dy = self.rng.randint(-self.radius, self.radius)
        dx = self.rng.randint(-self.radius, self.radius)

        if dx*dx + dy*dy > self.radius*self.radius:
            continue # Outside disk

        if self._is_valid_coord(ny, nx, H, W):
            neighbors.append({"y": ny, "x": nx, "label": lbl[ny, nx]})
    return neighbors

```

5.2 Samplers

BalancedBatchSampler

Ensures equal class representation in each batch.

```

class BalancedBatchSampler(Sampler):
    def __iter__(self):
        while num_batches_generated < self.max_batch:
            batch = []
            for label, indices in self.label_to_indices.items():
                if len(indices) < self.samples_per_label:
                    indices = random.choices(indices, k=max_class_size) # Oversample
                selected = random.sample(indices, self.samples_per_label)
                batch.extend(selected)
            yield batch

```

CombinedBatchSampler

Mix of balanced labeled + random unlabeled samples.

```

class CombinedBatchSampler(Sampler):
    def __init__(self, dataset, batch_size, labeled_ratio=0.50):
        self.small_batch_size = int(batch_size * labeled_ratio)

    def __iter__(self):
        # 50% balanced labeled
        balanced_batch = sample_balanced(self.label_to_indices, self.small_batch_size)
        # 50% random unlabeled
        random_unlabeled = random.sample(self.random_indices, remaining)
        yield balanced_batch + random_unlabeled

```

ModeAwareBalancedAnchorBatchSampler

Adapts batch composition based on dataset mode.

```

class ModeAwareBalancedAnchorBatchSampler(Sampler):
    def _compute_epoch_plan(self):
        if self.dataset.mode == "semisupervised":
            anchors_per_batch = self.total_patches_per_batch // 8 # 8 patches per group
        else:
            anchors_per_batch = self.total_patches_per_batch
        return anchors_per_batch, per_label_counts, num_batches

```

5.3 Data Loading

Collate Function

Custom collation for variable-size groups:

```
def flex_collate(batch):
    patches = torch.cat([b[0] for b in batch], dim=0) # [sum M, C, H, W]
    labels = torch.cat([b[1] for b in batch], dim=0) # [sum M]
    segs = torch.cat([b[2] for b in batch], dim=0) # [sum M, C, H, W]
    coords = torch.stack([b[3] for b in batch], dim=0)
    return patches, labels, segs, coords
```

6. Utility Functions (`lib/utils.py`)

Tensor Operations

```
def crop_img_tensor(x, size):
    """Crop tensor to target size (center crop)."""

def pad_img_tensor(x, size):
    """Pad tensor to target size (symmetric padding)."""

def normalize(img, mean, std):
    """Normalize: (img - mean) / std"""

def denormalize(img, mean, std):
    """Denormalize: img * std + mean"""
```

Data Augmentation

```
def augment_data(patches):
    """8x augmentation: 4 rotations + horizontal flip."""
    augmented = np.concatenate([
        patches,
        np.rot90(patches, k=1, axes=(-2, -1)),
        np.rot90(patches, k=2, axes=(-2, -1)),
        np.rot90(patches, k=3, axes=(-2, -1)),
    ])
    augmented = np.concatenate([augmented, np.flip(augmented, axis=-2)])
    return augmented
```

KL Divergence with Free Bits

```
def free_bits_kl(kl, free_bits, batch_average=False):
    """
    Prevent posterior collapse by ensuring minimum KL per layer.
    kl shape: (batch_size, layers)
    """
    if batch_average:
        return kl.mean(0).clamp(min=free_bits)
    return kl.clamp(min=free_bits).mean(0)
```

Metrics

```
def PSNR(gt, img, psnrRange):
    """Peak Signal-to-Noise Ratio."""
    mse = np.mean(np.square(gt - img))
    return 20 * np.log10(psnrRange) - 10 * np.log10(mse)
```

7. 3D Training Example

The file `examples/3D/Pixel_Noise/Training.py` demonstrates full 3D training.

Configuration

```
# Model architecture
num_latents = 3
z_dims = [32] * num_latents # 3 layers, 32 dims each
blocks_per_layer = 5
n_filters = 64
patch_size = 64
img_shape = (64, 64, 64)
conv_mult = 3 # 3D convolutions

# Loss weights
alpha = 1 # Reconstruction
beta = 1e-1 # KL divergence
gamma = 1e-2 # Contrastive

# Contrastive learning
mask_size = 5
contrastive_learning = True
margin = 250
lambda_contrastive = 0.5

# Training
batch_size = 16
lr = 3e-4
max_epochs = 500
```

Data Preparation

```
# Load 3D TIFF volumes
train_images = [tiff.imread(path) for path in train_img_paths]
train_labels = [tiff.imread(path) for path in train_lbl_paths]

# Normalize
data_mean = np.mean(np.concatenate([img.flatten() for img in train_images]))
data_std = np.std(...)
for idx in range(len(train_images)):
    train_images[idx] = (train_images[idx] - data_mean) / data_std

# Create datasets
train_set = CombinedCustom3DDataset(train_images, train_labels, labeled_indices)
train_sampler = CombinedBatchSampler(train_set, batch_size, labeled_ratio=1.0)
train_loader = DataLoader(train_set, sampler=train_sampler)
```

Model Instantiation

```
model = LadderVAE(
    z_dims=z_dims,
    blocks_per_layer=blocks_per_layer,
    data_mean=data_mean,
    data_std=data_std,
    noiseModel=None,
    conv_mult=3, # 3D
    device=device,
    batchnorm=True,
    free_bits=0.0,
    img_shape=img_shape,
    grad_checkpoint=True, # Save memory
    mask_size=mask_size,
    contrastive_learning=contrastive_learning,
    margin=margin,
    lambda_contrastive=lambda_contrastive,
).cuda()
```

Training Call

```
training.train_network(
```

```

        model=model,
        lr=lr,
        max_epochs=max_epochs,
        directory_path=directory_path,
        batch_size=batch_size,
        alpha=alpha, beta=beta, gamma=gamma,
        train_loader=train_loader,
        val_loader=val_loader,
        gaussian_noise_std=None,
        model_name=model_name,
        gradient_scale=256,
        use_wandb=True,
    )

```

8. Configuration Parameters

Complete Parameter Reference

Parameter	Type	Default	Description
Architecture			
`z_dims`	List[int]	`[32, 32, 32]`	Latent dimensions per layer
`blocks_per_layer`	int	`5`	Residual blocks per layer
`n_filters`	int	`64`	Base filter count
`conv_mult`	int	`2` or `3`	2D or 3D convolutions
`nonlin`	nn.Module	`nn.ELU`	Activation function
`dropout`	float	`0.2`	Dropout probability
`batchnorm`	bool	`True`	Use batch normalization
`gated`	bool	`True`	Use gating mechanism
`stochastic_skip`	bool	`True`	Skip around stochastic layers
`res_block_type`	str	`"bacdbacd"`	Residual block pattern
Training			
`lr`	float	`3e-4`	Learning rate
`batch_size`	int	`16`	Batch size
`max_epochs`	int	`500`	Maximum epochs
`alpha`	float	`1`	Reconstruction loss weight
`beta`	float	`1e-1`	KL loss weight
`gamma`	float	`1e-2`	Contrastive loss weight
`max_grad_norm`	float	`None`	Gradient clipping
`amp`	bool	`True`	Mixed precision
Latent Space			
`free_bits`	float	`0.0`	Minimum KL per layer
`learn_top_prior`	bool	`False`	Learnable top prior
`stochastic_block_type`	str	`"normal"`	`"normal"` or `"mixture"`
`n_components`	int	`4`	GMM components
Contrastive			

`contrastive_learning`	bool	`False`	Enable contrastive loss
`margin`	float	`50`	Contrastive margin
`lambda_contrastive`	float	`0.5`	Pos/neg loss balance
Inpainting			
`mask_size`	int	`5`	Center mask size
`initial_mask_size`	int	`1`	Curriculum start
`final_mask_size`	int	`10`	Curriculum end

9. Mathematical Foundations

Variational Lower Bound (ELBO)

The model maximizes:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q(\phi(z|x))} [\log p(\theta(x|z))] - D_{\text{KL}}(q_{\phi}(z|x) \parallel p(z))$$

Hierarchical Factorization

With L layers:

$$p(z) = p(z_L) \prod_{l=1}^{L-1} p(z_l | z_{>l})$$

$$q(z|x) = \prod_{l=1}^L q(z_l | z_{>l}, x)$$

KL Divergence (Diagonal Gaussian)

$$D_{\text{KL}}(q \parallel p) = \frac{1}{2} \sum_i \left[\log \frac{\sigma_{p,i}^2}{\sigma_{q,i}^2} + \frac{\sigma_{q,i}^2}{\sigma_{p,i}^2} + (\mu_q - \mu_p)^2 \sigma_{p,i}^2 - 1 \right]$$

Contrastive Loss

$$\mathcal{L}_{cl} = \mathbb{E}_{\text{pos}} [d(z_i, z_j)^2] + \mathbb{E}_{\text{neg}} [\max(0, m - d(z_i, z_j))^2]$$

where d is Euclidean distance and m is the margin.

Gumbel-Softmax (for discrete latents)

$$y_i = \frac{\exp((\log \pi_i + g_i)/\tau)}{\sum_j \exp((\log \pi_j + g_j)/\tau)}$$

where $g_i \sim \text{Gumbel}(0, 1)$ and τ is temperature.

10. File Structure

```
HDN/
├── README.md
├── requirements.txt
├── LICENSE.txt
├── training.py
├── Training_*.py
├──
├── models/
└── ─── __init__.py

# Project overview
# Dependencies
# License
# Main training loop
# Dataset-specific training scripts
```

```

■   ■■■ lvae.py                # LadderVAE main model
■   ■■■ lvae_layers.py        # Layer components
■
■■■ lib/
■   ■■■ nn.py                 # Residual blocks, gates
■   ■■■ stochastic.py         # Stochastic sampling blocks
■   ■■■ likelihoods.py        # Likelihood modules
■   ■■■ utils.py              # Utility functions
■   ■■■ histNoiseModel.py     # Histogram noise model
■   ■■■ gaussianMixtureNoiseModel.py # GMM noise model
■
■■■ boilerplate/
■   ■■■ boilerplate.py        # Training utilities
■   ■■■ dataloader.py         # Data loaders
■   ■■■ dataset.py            # Dataset classes
■
■■■ examples/
■   ■■■ 2D/
■       ■■■ Pixel_Noise/
■           ■■■ Convallaria/
■           ■■■ Training.py
■           ■■■ segmentation.py
■       ■■■ 3D/
■           ■■■ Pixel_Noise/
■               ■■■ Training.py
■               ■■■ segmentation.py
■
■■■ Prediction/
■   ■■■ test.py
■   ■■■ Eval_Metric.py
■   ■■■ Pred_*.py             # Dataset-specific prediction
■   ■■■ Compute_DSC_*.py     # Dice score computation

```

Summary

The HDN codebase provides a sophisticated implementation of hierarchical variational autoencoders for image denoising. Key innovations include:

1. **3D Extension:** Full support for volumetric data through flexible `conv_mult` parameter
2. **Semi-supervised Learning:** Efficient use of limited labeled data
3. **Contrastive Learning:** Multi-scale feature discrimination
4. **Curriculum Learning:** Gradual increase of task difficulty
5. **GMM Priors:** Richer latent space modeling

This architecture achieves state-of-the-art performance on multiple denoising benchmarks while providing interpretable latent representations.

Document Version: 1.0

Generated: February 2026

Repository: `3d_initial` branch